Library Search and Delivery Planner

CSD201 Assignment 2 | Fall 2026 | Class SE2072 | Group AHK

Implementation report

This standalone Java 8 console program combines a book index, an undirected weighted branch network, four hand-written sorting algorithms, and a repeatable comparison runner. It has no external dependencies. The data uses fictional branch names and distance units.

Design and validation

Area	Implementation	Input and edge behavior
Book index	Unbalanced BST keyed by lexicographic bookId	Reject duplicate IDs; preserve catalogue records
Branch network	Undirected adjacency list	BFS per reachable component; Dijkstra; Prim
Sorting	Quick, Merge, Heap, Insertion	One copies-then-bookId comparator
CSV loading	Three files parsed into temporary state	Replace state only after all files validate

The valid dataset contains 24 books, seven unique branches, and nine positive-distance routes. It is connected. Invalid example folders cover an unknown branch ID and a duplicate unordered route. Blank lines are ignored; malformed headers, column counts, identifiers, numbers, duplicate keys, self-loops, and invalid distances are rejected with the file and line number.

The loader constructs a candidate state from all three files before assigning it to the store. A failed load therefore leaves the previous state object untouched. A disconnected graph is accepted as valid data and is handled by the route and spanning-tree operations.

Run from the project directory with the commands in README.txt. The test runner and measurement generator operate on fresh arrays and do not alter the catalogue used by the console application.

A. Binary search tree

Each node stores one immutable Book. Insert and search compare bookId lexicographically; duplicates are rejected. Delete handles a leaf, a node with one child, and a node with two children by replacing the deleted node with its in-order successor. Inorder traversal emits ascending IDs. Node count is recursive.

Height convention and cost

Height is counted in nodes: empty tree = 0 and a leaf = 1. Operations take O(h), where h is the current tree height. An ordinary BST built in balanced order has height near log2(n); sorted insertion can make a degenerate chain of height n. No balancing rotations are used.

Deletion trace

Insert keys in this order: M, C, T, A, F, D, G. The notation key(left,right) omits empty children with a dot.

Operation	Before	Action	After
Delete leaf A	M(C(A,F(D,G)),T)	A has no children; unlink it	M(C(.,F(D,G)),T)
Delete one-child C	M(C(.,F(D,G)),T)	Promote C's only child F	M(F(D,G),T)
Delete two-child M	M(F(D,G),T)	Successor is minimum of right subtree: T	T(F(D,G),.)

The executable tests also verify an empty tree, singleton, duplicate rejection, inorder order, and a degenerate five-node chain with height 5. Deletion tests independently cover all three structural cases. The console reports the node-count height convention alongside its BST statistics.

B. Branch graph

The graph stores each undirected route in both endpoint adjacency lists, with its positive integer distance. Neighbors are visited in file order. BFS marks a branch when it is enqueued, so each reachable vertex appears once. Its cost is O(V + E). Dijkstra and Prim use linear array/map scans to select the next minimum key, giving O(V^2 + E) time and O(V + E) storage.

Supplied graph trace

Edges: A-B 4, A-C 1, C-B 2, B-D 1, C-D 5. With A as the source, BFS visits A, B, C, D. Adding isolated E does not add it to that traversal.

Dijkstra from A to D

Step	Chosen	Updated tentative distances
Initialize	A (0)	A=0, B=4, C=1, D=inf
1	C (1)	B=3 via C; D=6 via C
2	B (3)	D=4 via B
3	D (4)	Destination settled

Result: A -> C -> B -> D, total distance 4. For source=destination the result is the one-vertex path at distance 0. An unreachable destination prints NO ROUTE.

Prim minimum spanning tree

Chosen edge	Weight	Running total
A-C	1	1
C-B	2	3
B-D	1	4

The four-vertex graph has MST cost 4. After adding isolated E, A to E has no route and Prim returns NO SPANNING TREE with no partial edge list. Equal choices are deterministic: select the smallest branch ID among equal keys, and prefer the lexicographically smaller parent on equal edge weights.

C. Sorting implementations

All methods sort copies of the same input array using the total order (copies ascending, then bookId lexicographically ascending). The comparator is shared and every comparison in a sort calls the same counting wrapper. Input arrays are cloned before each run.

Algorithm	Average time	Worst time	Extra memory	Stable?
Quick Sort (last pivot)	O(n log n)	O(n^2)	O(log n) average recursion; O(n) worst	No
Merge Sort	O(n log n)	O(n log n)	O(n) temporary array	Yes
Heap Sort	O(n log n)	O(n log n)	O(1)	No
Insertion Sort	O(n^2)	O(n^2)	O(1)	Yes

Implementation notes

Quick Sort uses Lomuto partitioning with the last element as pivot; this makes already-sorted and reverse-sorted inputs quadratic. Merge Sort takes the left record on equal comparison, preserving order, and counts assignments into both its temporary array and the main array. Heap Sort builds and drains a max heap. Insertion Sort shifts only when the previous record compares greater, so equal keys retain their order.

Every swap performs and counts two record-array writes, including a self-swap. Insertion Sort counts each shift and final placement. Counts exclude array cloning, input generation, and verification. These counts describe this implementation, not all possible versions of each algorithm.

Sort correctness checks

The test runner checks empty, singleton, equal-copy-count records with distinct IDs, sorted, and reverse-sorted inputs for all four algorithms. It uses Java's Arrays.sort only as a test reference and also checks record preservation. The comparison runner uses a separate order-and-multiset checker, not a library sort.

D. Algorithm comparison

The generator tests sizes 100, 500, and 1,000 in seeded-random, already-sorted, reverse-sorted, and equal-copy-count orders: 4 algorithms x 3 sizes x 4 orders = 48 runs. The seed is 20261109; random inputs use java.util.Random with seed + size, unique IDs B00000 onward, and copies in [0,1000]. The equal-copy case assigns 42 copies and shuffles IDs with a size-dependent seed.

Correctness and test result

All 48 comparison rows report sorted=true and records_preserved=true. The dependency-free test runner reports 35 checks passed. Full per-run counts are in results/sorting.csv; timings are intentionally omitted.

Operation counts for n = 1,000

Algorithm	Input order	Comparator calls	Array writes
Quick	seeded random	11505	14332
Quick	already sorted	499500	1000998
Quick	reverse sorted	499500	500998
Quick	equal copy counts	10550	11164
Merge	seeded random	8680	19952
Merge	already sorted	5044	19952
Merge	reverse sorted	4932	19952
Merge	equal copy counts	8695	19952
Heap	seeded random	16887	18210
Heap	already sorted	17583	19416
Heap	reverse sorted	15965	16632
Heap	equal copy counts	16800	18080
Insertion	seeded random	247454	247464
Insertion	already sorted	999	999
Insertion	reverse sorted	499500	500499
Insertion	equal copy counts	256165	256173

The counters are deterministic. Merge Sort writes about 20,000 positions per 1,000-record run because each merge writes to a temporary array and copies back. The last-pivot Quick Sort performs 499,500 comparator calls on both sorted and reverse-sorted 1,000-record arrays, illustrating its quadratic worst case. Insertion Sort takes 999 comparisons and writes on sorted input, but grows sharply when inversions are common.

E. Findings and algorithm choice

For a nearly sorted catalogue, choose Insertion Sort when the number of records is moderate and edits disturb only a few positions. Its work is O(n + I), where I is the inversion count; it is stable and in-place. The 1,000-record sorted run needed 999 comparator calls and 999 writes. For a large unsorted catalogue, choose Merge Sort when predictable O(n log n) behavior and stability matter and O(n) temporary memory is available. Choose Heap Sort when auxiliary memory must stay constant; it gives O(n log n) worst-case time but is not stable. The measurements do not establish one algorithm as universally fastest.

Reproducibility and correctness

Use the README build command with a Java 8 JDK, then run TestRunner and CompareRunner. The test suite covers BST empty/singleton/duplicate/delete/inorder/degenerate cases; BFS reachability and duplicate visits; Dijkstra source=destination, unreachable, and competing routes; connected and disconnected MSTs; each sorter on five boundary/order cases; and transactional rejection of an invalid route file. The invalid-load check confirms the original state object remains in place and checks the routes.csv line number.

AI and review note

AI assistance was used to draft code and the report. One checked suggestion was the supplied shortest route and MST cost; TestRunner verifies A -> C -> B -> D at distance 4 and MST cost 4, then checks the isolated-vertex behavior. The group should review the code and report, and confirm actual member roles and contribution evidence before upload.

Package contents

The ZIP contains src/, valid and invalid data folders, results/ with the 48-row comparison CSV and test output, README.txt, report.pdf, members.csv, contributions.csv, and ai-use.txt. Do not add build output or IDE caches. The roster is supplied; the contribution sheet proposes an even module split and peer review cycle for the group to confirm or update to match its actual work.

End of report | Generated 9 November 2026